FraudGuard AI

Fraud Detection System - User Guide

Version 1.0 | June 2026

This guide covers the **FraudGuard AI Fraud Detection System** — a hybrid rule-based + machine-learning platform that analyses financial transactions in real time and flags or blocks suspicious activity.

---

1. Project Overview

FraudGuard AI combines **heuristic rules** (velocity checks, location anomalies, amount thresholds) with a **machine-learning model** to produce a risk score for every transaction. A web-based dashboard lets administrators configure security policies and review transaction logs in real time.

2. Technologies Used

Term	Definition
Python 3.10+	Core backend language; runs Flask API and ML inference.
Flask	Lightweight REST API server (app.py). Serves all /api/ endpoints.
MySQL	Relational database storing users, transactions, and security config.
HTML / CSS / JS	Frontend pages in the frontend/ directory (user, admin, transactions).
scikit-learn	ML library: trains Random Forest, Decision Tree, Logistic Regression.
pandas / numpy	Data manipulation and feature engineering for model training.
joblib	Serialises trained models to .pkl files for fast loading at runtime.
ReportLab	Python PDF library used to generate this document.

3. Key Definitions

Term	Definition
Velocity Rule	Detects >= 2 transactions from the same device within 60 s. Triggers an immediate HIGH RISK block

Term	Definition
Unusual Location	Transaction from a different country than the user's registered location. Adds +35 risk points (warning
High Amount Anomaly	Amount > $10,000. Adds +50 risk points and causes an immediate block.
Risk Score	Cumulative score from heuristic rules + ML probability. Score >= 100 = HIGH RISK.
Risk Levels	Low (0-49), Medium (50-99), High (>=100 blocked).
New Device Login	Transaction from a device ID not previously linked to the account. Shows a warning badge but does n
Blacklisted Account	Admin has manually banned the user ID. All transactions blocked immediately.
Int'l Payment Block	Admin security policy that blocks all transactions flagged as International.
fraud_model.pkl	Primary Random Forest model loaded by app.py for real-time inference.
model_features.json	JSON listing the exact feature columns expected by the ML model.
train_model.py	Script in ml/ that trains all three classifiers and saves .pkl files.
db_setup.py	Creates the MySQL schema and seeds default users and security settings.

4. Terminal Instructions (How to Run)

Prerequisites — Install & verify before starting

Ensure all of the following are installed and available in your terminal before proceeding.

Requirement	Version	Notes
Python	3.10 or newer	python --version
MySQL Server	8.0 or newer	Must be running before db_setup.py
pip	Latest	python -m pip install --upgrade pip
Git	Any	For cloning the repository

*NOTE: Make sure MySQL Server is **running** before you run db_setup.py. You can start it via MySQL Workbench, Windows Services, or: net start MySQL80*

Step 1 - Clone the Repository

Download the project source code from GitHub and navigate into the project folder.

```
git clone https://github.com/techie-suhaib/Fraud-detection-using-ML.git
cd "Fraud Detection using ML"
```

Expected output: A new folder 'Fraud Detection using ML' is created with all project files inside.

Step 2 - Create a Virtual Environment (Recommended)

Isolates project dependencies from other Python projects on your system.

```
# Create the virtual environment
python -m venv venv

# Activate it (Windows PowerShell)
.\venv\Scripts\activate
```

```
# Activate it (Windows CMD)
venv\Scripts\activate.bat

# Activate it (macOS / Linux)
source venv/bin/activate
```

Expected output: Your terminal prompt will show (venv) when the environment is active.

Step 3 - Install Python Dependencies

Install all required packages in one command.

```
python -m pip install --upgrade pip
python -m pip install flask mysql-connector-python scikit-learn pandas numpy joblib rep
ortlab
```

Expected output: All packages install without errors. Verify with: pip list

Step 4 - Configure the .env File

The project reads database credentials and the Flask secret key from a **.env** file in the project root. Open it and update the values to match your MySQL setup.

```
# .env  (already exists — edit the values below)
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=your_mysql_password
DB_NAME=fraud_detection_db

SECRET_KEY=fraud_guard_ai_secret_key_123!@#
```

NOTE: Change DB_PASSWORD to your actual MySQL root password. DB_NAME will be created automatically by db_setup.py — do not create it manually.

Step 5 - Train the Machine Learning Models

Run the training script **before** starting the Flask server. It generates three model (.pkl) files inside the **ml/** folder that the API loads at startup. If no CSV dataset is present it automatically synthesises 50,000 realistic transactions for training.

```
python ml/train_model.py

# --- Optional: use a real PaySim dataset ---
# Place the CSV file at  ml/new_file.csv  then run the same command.
# The script will load it automatically (first 500,000 rows).
```

Expected output: Training completes and you see:

```
ML model 'random_forest' loaded successfully.
ML model 'decision_tree' loaded successfully.
ML model 'logistic_regression' loaded successfully.
All model training tasks completed successfully!

# Files created inside ml/:
#   random_forest.pkl      <- primary model used by app.py
#   decision_tree.pkl
#   logistic_regression.pkl
#   model_features.json    <- feature list read by app.py
```

Step 6 - Initialise the Database

Creates the **fraud_detection_db** MySQL database, all required tables, and seeds default admin/user accounts and security settings. Run this **once** (re-running is safe — existing data is preserved).

```
python db_setup.py
```

Expected output: Database and tables created. Default users seeded successfully.

NOTE: If you see 'Access denied' — check that DB_PASSWORD in .env is correct and that your MySQL server is running.

Step 7 - Start the Flask Server

Launch the application. Flask will load all three ML models and connect to MySQL on startup.

```
python app.py
```

Expected output: Server starts with no errors:

```
ML model 'random_forest' loaded successfully.
ML model 'decision_tree' loaded successfully.
ML model 'logistic_regression' loaded successfully.
Features list loaded successfully.
 * Running on http://127.0.0.1:5000
 * Debug mode: on
```

Open a browser and navigate to the pages below:

```
Admin Dashboard -> http://127.0.0.1:5000/admin.html
User Dashboard  -> http://127.0.0.1:5000/user.html
Transactions    -> http://127.0.0.1:5000/transaction.html
```

5. Default Login Credentials

Term	Definition
Admin	admin@fraudguard.ai / admin123 -> http://127.0.0.1:5000/admin.html
Regular User	user@test.com / user123 -> http://127.0.0.1:5000/user.html

6. Admin Setup (Security Policies)

1. Open **http://127.0.0.1:5000/admin.html** and log in with **admin@fraudguard.ai / admin123**.
2. Click the **Security** link to open *security.html*.
3. Toggle **Block International Payments** ON.
4. In **Account Blacklist**, type **2** (test user ID) and click **Ban Account**.
5. Click **Save Changes** — a "Changes Saved!" toast confirms the update.
6. Logout (top-right) to return to the login screen.

7. Verification Scenario Table

Use the Location and Device selectors in the transaction form for each scenario. Complete admin setup (Section 6) before running scenarios 6 and 7.

#	Amount ($)	Location	Device	Expected Outcome
1	150	Local	Same Device (Linked)	**Transaction Safe**
2	200	Local	New Device (Unlinked)	**Transaction Safe** **+ New Device Login badge**
3	10->10->50 (same device)	Local	Same Device (Linked)	**HIGH RISK BLOCKED** **(High Velocity)**
4	12,000	Local	Same Device (Linked)	**HIGH RISK BLOCKED** **(High Amount Anomaly)**

#	Amount ($)	Location	Device	Expected Outcome
5	100	International	Same Device (Linked)	**Transaction Safe (Unusual Location warning)**
6	100	International	Same Device (Linked) after enabling Block Int'l Payments	**HIGH RISK BLOCKED (Int'l Payment Blocked)**
7	50	Local	Same Device (Linked) after blacklisting user 2	**HIGH RISK BLOCKED (Blacklisted Account)**

8. Viewing the Transaction Log

After running the scenarios, open **http://127.0.0.1:5000/transaction.html** and verify each row shows the correct **Risk Level** and **Status** badge.